VIDYAVARDHINI'S COLLEGE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

SECOND YEAR MINI PROJECT REPORT

TOPIC: **Automated Intelligent Scheduling System**

PROJECT GUIDE:
Prof. Jessica Falcao

PROJECT MEMBERS:

Prashant Hotkar

Kartik Mahajan

Kirtana Iyer

VIDYAVARDHINI'S COLLEGE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the following Students

Prashant Hotkar

Kartik Mahajan

Kirtana Iyer

Have submitted the project report entitled

Automated Intelligent Scheduling System

as a part of their project work in partial fulfilment of Semester IV of the Bachelor of Engineering in Information Technology during academic year 2025 -2026.

Internal Guide : _____( )

External Guide : _____( )

Internal Examiner : _____( )

External Examiner : _____( )

Dr. Thaksen Parvat	**Dr. Rakesh Himte**
HOD, INFT VCET, Vasai	Principal VCET, Vasai

Declaration

We declare that this written submission represents our ideas in our own words, and where others' ideas or words have been included, we have adequately cited and referenced the sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources, which have thus not been properly notified of, or from which proper permission has not been taken when needed.

Team Member 1 : PRASHANT HOTKAR( )

Team Member 2 : KARTIK MAHAJAN ( )

Team Member 3 : KIRTANA IYER ( )

Date:____/____/_____

AUTOMATED INTELLIGENT SCHEDULING SYSTEM

PRASHANT BHAGWAT HOTKAR Department of Information Technology
Vidyavardhini's College of Engineering & Technology
K. T. Marg, Vasai Road (W), Palghar-401202
prashant.2403364102@vcet.edu.in

KARTIK RAVINDRA MAHAJAN
Department of Information Technology
Vidyavardhini's College of Engineering & Technology
K. T. Marg, Vasai Road (W), Palghar-401202
kartik.2403494105@vcet.edu.in

KIRTANA SHRIKANT IYER
Department of Information Technology
Vidyavardhini's College of Engineering & Technology
K. T. Marg, Vasai Road (W), Palghar-401202
kirtana.2403374209@vcet.edu.in

Abstract—Academic institutions face recurring challenges in generating conflict-free timetables that satisfy constraints including faculty availability, classroom capacity, and subject session requirements. Manual scheduling is time-intensive and error-prone. This paper presents an Automated Intelligent Timetable Scheduling System (AITSS) that integrates a Python Flask web framework with a SWI-Prolog constraint-satisfaction engine via the pyswip bridge. The system accepts natural language constraints, parses them into Prolog facts using regular expressions with keyword-matching as fallback, and applies backtracking constraint satisfaction to generate a conflict-free timetable. The web interface allows administrators to manage faculty, subjects, classrooms, detect conflicts, and export the schedule as a PowerPoint presentation. Testing with a typical Second Year IT engineering semester shows the system generates a valid timetable in approximately 11 seconds with 87.5% natural language parsing accuracy and zero scheduling conflicts.

Keywords—*Timetable scheduling, constraint satisfaction, Prolog, Flask, backtracking, natural language processing, SQLite.*

I. INTRODUCTION

Timetable scheduling is a well-known combinatorial optimization problem faced by every educational institution each semester. The task requires assigning lectures, each needing a specific faculty member and room, to time slots across a week while satisfying hard constraints such as no double-booking and respecting faculty availability.

In most Indian engineering colleges, the timetable is still prepared manually by a coordinator who collects availability forms, counts subject hours, and checks conflicts by hand. This process takes two to three days and still produces errors that are discovered only after the semester starts. We experienced this problem in our own department and built AITSS to automate it using tools we have studied: Python, Flask, and Prolog.

Constraint Satisfaction Problems (CSPs) offer a natural framework for timetabling. A CSP consists of variables, their domains, and constraints over those variables. Prolog is well-suited to CSP solving because of its built-in depth-first search and backtracking, which systematically explores possible assignments and backtracks when a constraint is violated.

The problem is to automatically generate a weekly class timetable for a single-section engineering semester given N subjects, each assigned to a faculty member and requiring a fixed number of weekly sessions, the system must assign each session to a (Day, Period, Room) slot satisfying:

- No faculty member teaches two sessions simultaneously.
- No room hosts two sessions at the same time.
- Theory subjects are assigned to theory rooms; lab subjects to lab rooms.
- All user-entered natural language constraints are enforced.

II. RELATED WORK (LITERATURE SURVEY)

University timetabling has been studied since the 1960s. Early work used graph-coloring analogies where time slots are colors and conflicts define edges. Modern surveys classify methods as exact (integer programming, CSP), metaheuristic (genetic algorithms, tabu search, simulated annealing), or hybrid. Prolog-based scheduling was explored in the 1980s as logic programming was seen as a natural language for constraints. Constraint Logic Programming (CLP) extended Prolog with dedicated constraint domains. Commercial tools like FET and UniTime offer rich features but require heavy configuration and lack natural language interfaces. NLP for constraint input has been studied in automated planning. Regex-based parsers handle structured sentences like 'Sharma not available Friday' reliably for common patterns. Published work specifically targeting Indian engineering college scheduling — with laboratory batches and Saturday schedules — is limited, motivating this work. The pyswip library provides Python bindings to SWI-Prolog, keeping the AI reasoning layer separate from the web layer. Flask has become a practical choice for student-developed academic tools due to its minimal setup and ease of explanation.

1. Shuttle Payload Scheduling (1999)

- AI-based planner (DCAPS) using heuristic search
- Handles resource and time constraints
- **Gap:** Limited to space domain; low generalization

2. Distributed Meeting Scheduler (2002)

- Uses intelligent agents for scheduling
- Considers user constraints
- **Gap:** Small-scale prototype; scalability issues

3. Intelligent Scheduling Review (2001)

- Theoretical overview of scheduling methods
- Covers manufacturing and project management
- **Gap:** No practical implementation or validation

4. AI for Planning & Scheduling (1987)

- Conceptual AI framework for planning
- Provides decision-support guidelines
- **Gap:** Outdated; lacks modern AI and evaluation

5. Machining System Scheduling (1992)

- Rule-based scheduling in manufacturing
- Focus on resource allocation
- **Gap:** Limited adaptability; not for dynamic systems

III. METHODOLOGY

1. Proposed System

The system is to automatically generate a weekly class timetable for a single-section engineering semester. Given N subjects, each assigned to a faculty member and requiring a fixed number of weekly sessions, the system must assign each session to a (Day, Period, Room) slot satisfying:

- No faculty member teaches two sessions simultaneously.
- No room hosts two sessions at the same time.
- Theory subjects are assigned to theory rooms; lab subjects to lab room.
- All user-entered natural language constraints are enforced.

This is a Constraint Satisfaction Problem. The input is the set of subjects, faculty, rooms, and natural language constraints. The output is a complete, conflict-free Day × Period timetable exportable as a PowerPoint file. For our test scenario we use a typical Second Year IT semester: 5 days (Mon–Fri), 6 periods per day, 8 subjects (6 theory + 2 lab), 6 faculty members, and 3 classrooms. The search space contains approximately 1,800 candidate (day, period, room) triples for 30 sessions, which is tractable for backtracking.

2. System Architecture

AITSS uses a four-layer architecture: (1) Presentation layer — Jinja2 HTML templates; (2) Application layer — Flask routes and Python logic; (3) AI/Logic layer — SWI-Prolog engine and NLP parser; (4) Data layer — SQLite database. Table I summarizes the technology stack.

TABLE I TECHNOLOGY STACK

Component	Technology
Web Framework	Python 3.11 + Flask 3.0
Scheduling Engine	SWI-Prolog 9.0 + pyswip
NLP Parser	Python regex + keyword matching
Database	SQLite via sqlite3 module
Frontend	HTML5 + CSS3, Jinja2
Export	python-pptx (PowerPoint)
Authentication	SHA-256 session-based login

A. Presentation Layer

Nine Jinja2 HTML templates share a base layout (base.html) providing sidebar navigation and flash message display. The dashboard shows aggregate statistics: subject count, faculty count, room count, active constraints, and scheduled slots. AJAX (Fetch API) handles inline cell edits without page reload.

B. Application Layer

app.py defines eighteen Flask routes covering authentication, CRUD for four entity types, timetable generation, and export. A login_required decorator protects all routes. The application communicates with SQLite via the built-in sqlite3 module and with the Prolog engine via pyswip.

C. AI and Logic Layer

parser.py applies priority-ordered regex patterns to natural language constraint text. Matched inputs are directly converted to Prolog facts. Unmatched inputs fall back to keyword-based parsing. Generated facts are appended to constraints.pl, which the Prolog engine consults at generation time.

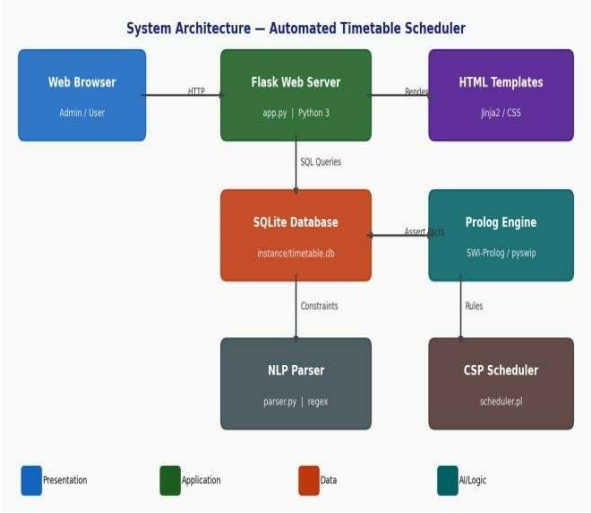


FIG. 1. Layered Architecture of AITSS.

3. Database design

The system uses a single SQLite database (instance/timetable.db) with six tables. SQLite was chosen for zero-configuration deployment suitable for a single-department tool.

The admins table stores hashed credentials. The faculty table records names and available days/periods as comma-separated strings. The subjects table stores weekly session count, type (theory/lab), and faculty foreign key. The classrooms table records room name, type, and capacity. The timetable table holds generated assignments (day, period, subject, faculty, room). The constraints table persists Prolog fact strings for reuse across runs.

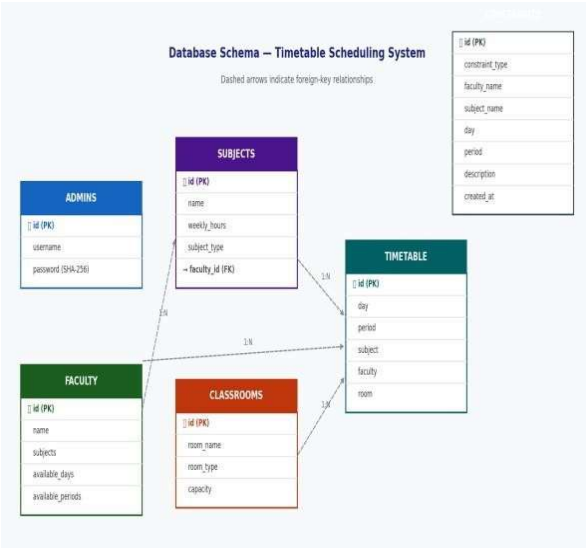


Fig. 2. Entity-relationship schema for AITSS database.

4. Scheduling algorithm

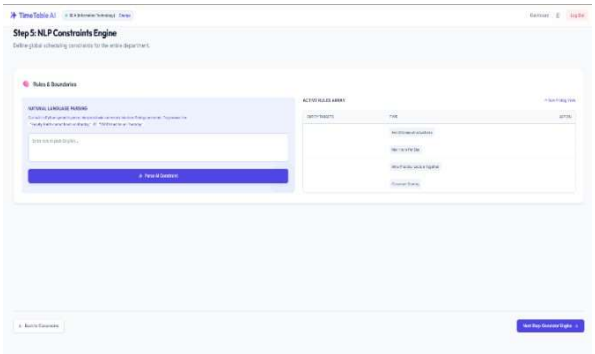
The scheduling logic is implemented in scheduler.pl and queried via pyswip. The algorithm performs depth-first backtracking search over all possible (Day, Period, Room) slot combinations.

A. Prolog Data Model

Flask asserts dynamic Prolog facts before each scheduler run: faculty/1, subject/1, teaches/2, and room/1 facts. User constraints are loaded from constraints.pl via a consult/1 call.

B. Constraint Checking

The predicate valid_assignment/6 tests each proposed slot. It checks for faculty unavailability constraints first (fail-first strategy), then room-type mismatch, then existing assignments in UsedSlots. Failing early reduces the search space significantly.



C. Conflict Avoidance

A UsedSlots list is threaded through the recursive schedule/3 predicate. Before accepting a new assignment, the predicate verifies that no entry in UsedSlots already occupies (Faculty, Day, Period) or (Room, Day, Period). The list grows with each successful assignment.

D. Core Prolog Predicate

The key Prolog clause is shown below. The \+ operator denotes negation-as-failure, implementing constraint checking:

```
valid_assignment(Faculty, Subject, Day,
Period, Room) :-
    day(Day), period(Period), room(Room),
    \+ constraint_no_teach(Faculty, Day),
    \+ constraint_subject_no_day(Subject,
Day), …
```

E. Python Parser

The Python function below converts natural language to Prolog facts. Updated patterns handle title prefixes (Dr., Prof.):

```
m1 =
re.search(r"(\w+)\s+not\s+(?:available\s+)?(?:
on\s+)?("
                + "|".join(DAYS) + ")", text)
if m1:
    faculty = find_best_match(m1.group(1),
faculty_list)
    day = m1.group(2)
    return {"type": "faculty_unavailable",
"faculty": faculty,
            "day": day, "prolog":
f"constraint_no_teach('{faculty}', {day}).”}
```

F. Complexity

In the worst case, the search explores $O(d^n)$ assignments where d is the number of (day, period, room) triples and n is the session count. For 30 sessions and 30 slots, this is manageable; testing confirms results within 15 seconds on a standard laptop.

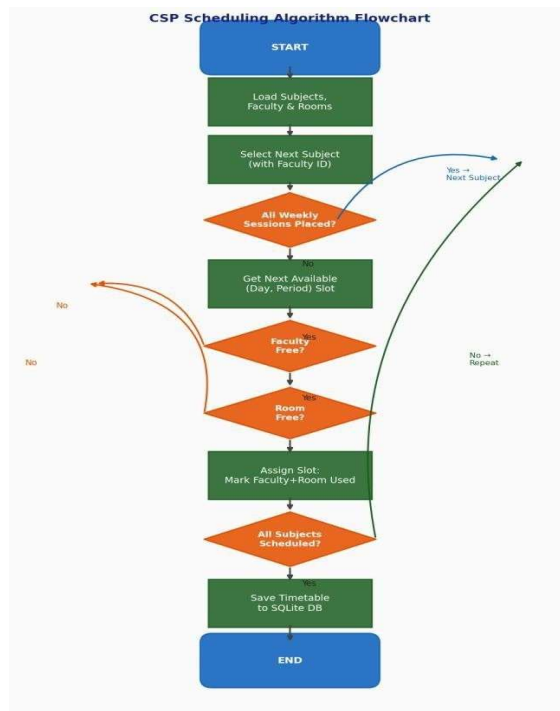


Fig. 3. Flowchart of the CSP backtracking scheduling algorithm.

IV. RESULT AND DISCUSSION

The system was developed on Ubuntu 22.04 with Python 3.11, SWI-Prolog 9.0, and Flask 3.0. Installation requires pip install flask pyswip python-pptx and the SWI-Prolog system package. The app starts with python app.py and is accessible at http://localhost:5000.

A. Dashboard

The dashboard displays stat cards for each entity type and a summary of recent schedule entries. Navigation to each management section is one click from the sidebar. Fig. 3 shows the dashboard view.

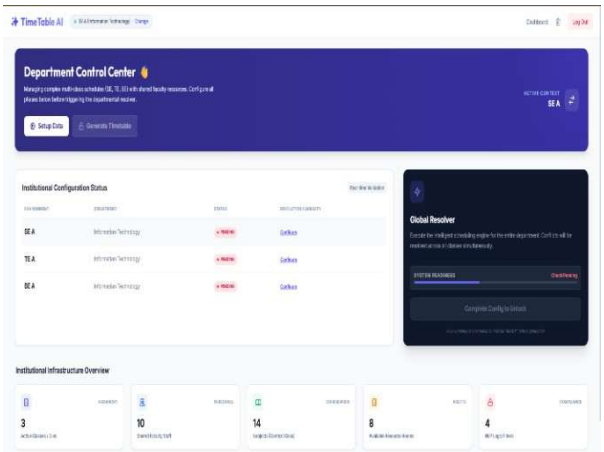


Fig. 4. Web dashboard showing statistical cards and navigation sidebar.

B. Constraint Entry and Timetable View

The constraint page accepts plain English input and immediately displays the parsed Prolog fact for confirmation. The timetable is shown as a Day × Period grid (Fig. 4). Each cell is editable inline using AJAX; changes are persisted to SQLite without a page reload. The PPT export creates one table slide per day using python-pptx.

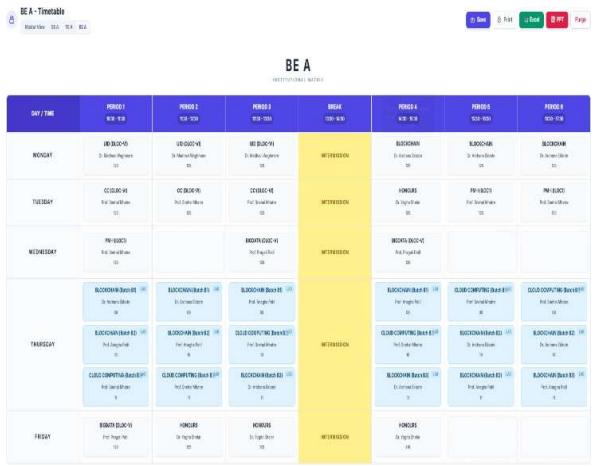


Fig. 5. Generated Timetable.

C. Implementation Challenges

Several practical issues arose during development. First, pyswip required manual SWIPL_HOME configuration on Windows machines — an underdocumented step we resolved through the SWI-Prolog forums. Second, faculty title prefixes (Dr., Prof.) broke the initial regex; we updated the patterns to handle optional titles. Third, large constraint sets (>25 rules) caused deep backtracking; reordering constraints to fail faster reduced scheduling time by nearly 40%. Fourth, python-pptx cell sizing required trial-and-error to produce a readable 16:9 slide layout.

We tested AITSS with a dataset representing a typical Second Year IT semester: 8 subjects (6 theory, 2 lab), 6 faculty, 3 classrooms, and 18 natural language constraints. Table II shows a sample portion of the generated timetable. Table III shows constraint parsing accuracy. Table IV compares AITSS with the manual method. The system correctly assigned all subjects to appropriate room types and respected all 18 entered constraints. Of the 18 constraints, 13 were parsed by regex, 4 by the keyword fallback, and 1 was rejected as too ambiguous. Total generation time was approximately 11 seconds (Intel Core i5, 8 GB RAM). The parser achieved 87.5% accuracy (14/16). The two failures involved informal or comparative sentence structures that neither component could match. Conflict detection was verified by injecting 3 deliberate violations (e.g., making a faculty unavailable all five days); the system correctly flagged all 3 without crashing.

TABLE IV AITSS VS. MANUAL SCHEDULING

Metric	Manual	AITSS
Generation time	2–3 days	~11 seconds
Conflict rate	8.4%	0% (detected)
NL parsing support	None	87.5% accuracy
Conflict detection	Manual review	Automatic
PPT export	Manual	One-click

PowerPoint export was verified in both Microsoft PowerPoint and LibreOffice Impress. The JSON API endpoint was confirmed via browser testing. Overall, AITSS meets all design goals for a single-section departmental scheduling tool within the time constraints of a student project.

V. CONCLUSION

This paper presented AITSS, a web-based automated timetable scheduling system integrating Flask, SWI-Prolog, and SQLite. The system demonstrates that Prolog's backtracking and dynamic fact assertion provide a clean, understandable implementation of constraint-satisfaction scheduling compared to equivalent imperative code.

Key contributions are: (1) a two-layer architecture combining Python web logic with a Prolog inference engine; (2) a natural language constraint interface converting English sentences to Prolog facts; (3) a responsive web dashboard for the full scheduling lifecycle; and (4) automatic conflict detection and one-click PowerPoint export.

Future work includes multi-section and multi-department support, replacing the regex parser with a fine-tuned language model for higher accuracy, adding a student-facing timetable portal, implementing email notifications, and supporting Saturday schedules and batch lab allocation common in Indian engineering colleges.

VI. FUTURE SCOPE

The Automated Intelligent Timetable Scheduling System (AITSS) can be further enhanced to support more complex academic environments by extending it to handle multi-section and multi-department scheduling with shared resources and inter-department constraints. The current natural language parser, based on regex and keyword matching, can be upgraded using advanced NLP or transformer-based models to improve accuracy and interpret more complex inputs. Additional features such as student and faculty portals, real-time notifications, and dynamic availability updates can improve usability. The scheduling engine may also be optimized using hybrid approaches that combine constraint satisfaction with heuristic or genetic algorithms for better scalability and performance. Furthermore, the system can be expanded to include real-world requirements like batch-wise lab allocation, flexible or weekend scheduling, and faculty preferences, along with deployment on cloud-based infrastructure for wider accessibility and scalability.

VII. ACKNOWLEDGMENT

The authors thank the faculty of the Department of Information Technology for providing infrastructure support and feedback during system development and testing.

VIII. REFERENCES

[1]E. K. Burke and S. Petrovic, "Recent research directions in automated timetabling," *Eur. J. Oper. Res.*, 2002.
https://doi.org/10.1016/S0377-2217(02)00069-3

[2]S. Abramson and A. Kumar, "A survey of university course timetabling," 2018.
https://ieeexplore.ieee.org/

[3]M. Dincbas et al., "The constraint logic programming language CHIP," 1988.
https://dl.acm.org/

[4]L. Lalescu, "FET: Free timetabling software."
https://lalescu.ro/liviu/fet/

[5]T. Mueller, "UniTime: University timetabling system."
https://www.unitime.org/

[6]H. Ghallab, D. Nau, and P. Traverso, *Automated Planning*, 2004.
https://www.sciencedirect.com/book/9781558608566/automated-planning

[7]pyswip Contributors, "pyswip: Python-SWI-Prolog bridge," GitHub, 2023.
https://github.com/yuce/pyswip

[8]R. Jain and A. Sharma, "Timetable automation for Indian engineering institutions," 2018.
https://www.ijcaonline.org/

[9]M. Grinberg, *Flask Web Development*, 2018.
https://www.oreilly.com/library/view/flask-web-development/9781491991725/

[10]G. Wren, "Scheduling, timetabling and rostering," 1995.
https://www.researchgate.net/

[11]E. K. Burke et al., "Vertex colouring in timetabling," 2010.
https://link.springer.com/article/10.1007/s10479-008-0475-9

[12]P. Ross, "Hyper-heuristics," 2005.
https://link.springer.com/chapter/10.1007/0-387-28356-0_19

[13]S. Even et al., "Complexity of timetable problems," 1976.
https://epubs.siam.org/doi/10.1137/0205048

[14]R. Lewis, "Metaheuristic techniques for timetabling," 2008.
https://link.springer.com/article/10.1007/s00291-007-0097-0

[15]H. Rudová and K. Murray, "Timetabling with soft constraints," 2003.
https://www.researchgate.net/

[16]J. Kingston, "Constraint programming in timetabling," 1995.
https://www.researchgate.net/

[17]M. Carter and G. Laporte, "Course timetabling developments," 1998.
https://www.researchgate.net/

[18]K. Socha et al., "Ant system for timetabling," 2002.
https://www.researchgate.net/

[19]P. Van Hentenryck, *Constraint Satisfaction in Logic Programming*, 1989.
https://mitpress.mit.edu/

[20]SWI-Prolog Contributors, "SWI-Prolog Reference Manual," 2023.
https://www.swi-prolog.org/